# DLSense: Distributed Learning-based Smart Virtual Sensing for Precision Agriculture

Rituparna Saha, Aishwariya Chakraborty, *Student Member, IEEE,* Sudip Misra, *Senior Member, IEEE,* Sajal K. Das, *Fellow, IEEE* and Chandranath Chatterjee, *Senior Member, IEEE*

***Abstract*— This work presents the design of an efficient edge-empowered sensor-cloud architecture equipped with a smart virtual sensing scheme for precision agriculture. Traditionally, in agricultural sensor-cloud, sensor nodes send raw sensed data periodically to the cloud, resulting in higher latency and higher energy and bandwidth consumption. The environment-dependent nature of agricultural parameters also limits the serviceability of sensor-cloud in regions with damaged or unemployed sensors. Moreover, the agricultural sensor-cloud suffers from privacy issues due to the sharing of sensitive farming data across third-party service providers. To address these drawbacks, we first propose a modified sensor-cloud architecture using edge devices as the middleware layer for sensor virtualization, thereby reducing service provisioning latency and resource consumption. Next, we propose *DLSense*, a novel intelligent virtualization scheme to aid in the design of virtual sensors in the absence of working sensor nodes in a region. DLSense utilizes correlation theory and distributed learning in the edge devices to predict sensor data and enables sharing of information of the trained models instead of raw sensed data, thus imparting privacy. Finally, we evaluate the performance of the DLSense scheme through extensive simulations and an experimental case study of an agricultural application. Results demonstrate that our proposed scheme reduces latency and service cost by 81% and 66%, respectively, and increases service availability by 39% compared to the state-of-the-art methods.**

***Index Terms*— Sensor Virtualization, Sensor Cloud, Edge intelligence, Correlation, Distributed Learning**

## I. Introduction

In recent years, the use of smart sensing technologies in *precision agriculture* (PA) has gained significant attention due to their applications to automated irrigation scheduling, crop health monitoring, greenhouse gas monitoring, and so on [1]–[3]. These technologies help measure various agricultural parameters, and enable farmers to make optimum decisions to improve the quality and productivity of the crops. Therefore, different kinds of wireless sensor nodes and smart devices are widely deployed to measure soil-related parameters (e.g., moisture, salinity, and conductivity), environment-related parameters (e.g., humidity, ambient temperature, and rainfall), and plant-related parameters (e.g., moisture, temperature, and $CO_2$). However, wireless sensor nodes are highly resource constrained in terms of computation and bandwidth capability, battery power, and storage capacity. This necessitates the development of resource-efficient sensor architectures, algorithms, protocols, and systems to analyze different parameters in PA for efficient information sharing in privacy-preserving manner. Moreover, effective PA applications require the farmers to install wireless sensor network (WSN) infrastructure on their fields, which incurs significant costs.

To address the aforementioned challenges, the concept of virtualization of sensor nodes has been proposed in the so called *Sensor Cloud (SC)* architecture for service provisioning [4]. SC utilizes cloud computing to enhance the limited capabilities of WSNs. In SC, sensor owners deploy sensor nodes and lease them to a (third-party) service provider that manages the network and cloud infrastructure as well as takes care of the maintenance overheads. The users (e.g., farmers) utilize sensor-based services through a web interface [5]. Although SC facilitates PA applications by reducing infrastructure cost and increasing scalability and fault tolerance, there still arise several unresolved issues as explained below.

(1) In SC, the selected sensor nodes within a region of interest send enormous amount of raw data either periodically or in an event-driven manner to the cloud, leading to increased *energy consumption and latency.*

(2) The *context-aware nature of sensor data* limits the serviceability of the SC in the regions with unemployed or damaged sensor nodes. Thus, in the absence of adequate sensor nodes, the service provider may be unable to provision for the corresponding services as desired by the end users.

(3) Though the SC provides various levels of abstraction of data across end users (farmers), it suffers from *privacy* risks. In the agriculture domain, the end user data are often sensitive as they include information about farmers, crop yield, profitability, etc. Moreover, such raw data need to be transmitted over large distances to the cloud using the SC network, thereby

Rituparna Saha and Aishwariya Chakraborty are with the Advanced Technology Development Centre, Indian Institute of Technology Kharagpur, India. Email: {rituparnasaha.official, aishwariya.chakraborty}@gmail.com

Sudip Misra is with the Department of Computer Science and Engineering, Indian Institute of Technology Kharagpur, India. Email: smisra@cse.iitkgp.ac.in

Sajal K. Das is with Department of Computer Science, Missouri University of Science and Technology, Rolla, USA. Email: sdas@mst.edu

Chandranath Chatterjee is with the Department of Agriculture and Food Engineering, Indian Institute of Technology Kharagpur, India. Email: cchatterjee@agfe.iitkgp.ac.in

For Personal Use Only

increasing security risks. Hence it is important to design privacy-aware and resource-efficient solutions that overcome the drawbacks of using SC for precision (smart) agriculture.

Alongside, edge computing empowered with Artificial Intelligence (AI) is being recently adopted to efficiently serve real-time applications that generate enormous amounts of data and require quick decision-making. For example, the authors in [6], [7] proposed the use of AI-empowered edge for WSN-based applications with strict latency demands. This led to the conceptualization of a new technology, termed as *Edge Intelligence* [8], [9], which is targeted towards bringing intelligence (computation and analysis) closer to the data generating devices, thereby decreasing latency. We envision that the integration of an intelligent edge layer with the traditional agricultural SC can help to overcome various aforementioned drawbacks. This motivated us to propose an enhanced SC architecture equipped with a *novel virtual sensing scheme*, called DLSense, which improves the performance of precision agriculture (PA) applications by introducing edge intelligence into the architecture (see Figure 1).

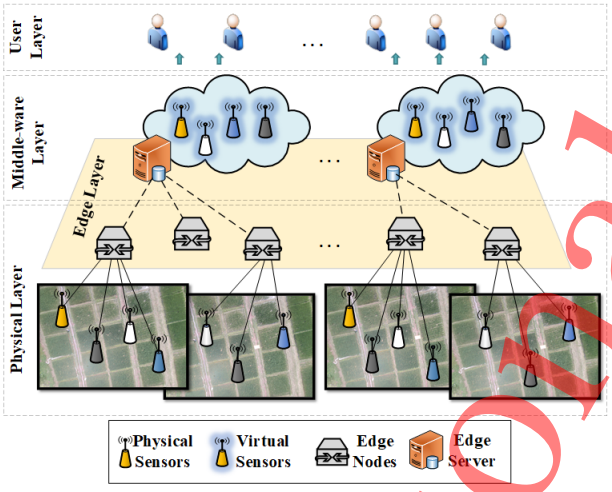


Fig. 1: Proposed DLSense based Sensor Cloud Architecture

***An Example Scenario:*** Let us consider two agricultural fields, say Field 1 and Field 2, with varying demography and topology in the agricultural region. We assume that the sensor owners have deployed WSNs in both the fields to serve the same application, say irrigation scheduling, for the respective farmers. Further, we consider that the nodes deployed in Field 1 are equipped with three types of sensors – humidity, temperature, and soil moisture; while the nodes deployed in Field 2 are equipped with only humidity and temperature sensors. Using traditional SC, it is not possible to obtain the soil moisture data of Field 2. Additionally, the soil moisture data of Field 1 cannot be used to serve irrigation scheduling in Field 2 due to the context-awareness of the data. However, using our proposed DLSense architecture, it is possible for a service provider to generate the machine learning (ML) model to predict soil moisture data from the available sensors in Field 1 and use in Field 2. This helps in increase the service availability and profits, while ensuring low latency and higher privacy by removing the need to share raw sensor data.

Our major contributions in this work are as follows.

- ***Integration of Edge Intelligence into SC:*** We propose a sensor cloud (SC) architecture equipped with an intelligent edge layer for PA. Majority of the computations and processing or analytics take place in the edge network, including sensor virtualization, thus reducing energy consumption and delay.
- ***Novel Virtualization Scheme:*** We design a novel virtual sensor formation scheme, called *DLSense*, with the help of a distributed learning framework. The proposed scheme provides support for active, inactive, and latent (damaged or unemployed) sensor nodes, and improves privacy through sharing of the ML models instead of raw data.
- ***Performance Analysis:*** Through extensive simulation study, we evaluate the performance of the proposed architecture equipped with DLSense in terms of energy consumption, latency, and message overheads; and compare them with the existing scheme. Results demonstrate that the proposed architecture reduces latency and cost by 81% and 66%, respectively; and also increases service availability by 39%.
- ***Use-case Study:*** We present a simple case study of a PA application using the proposed architecture.

The paper is organized as follows. Section II reviews the related literature. Sections III and IV present the proposed DLSense based sensor cloud architecture and virtualization scheme, respectively. Section V evaluates the performance of the proposed system, while Section VI concludes the paper.

## II. Related Work

This section reviews the related existing literature, categorizing them into works focusing on (a) WSN and SC for PA applications, and (b) virtualization of sensor nodes.

**(a) WSN and SC for PA Applications:** While WSNs help improve the traditional farming methods in various ways, to overcome the challenges resulting from the resource constrained nature of WSNs, researchers have focused on utilizing sensor cloud (SC) framework [4] for PA applications. The authors in [18] discussed the details of SC architecture, various research issues and design challenges, as well as benefits of real-time applications. Tyagi *et al.* [19] proposed a measurement to management system, named "PI-Cloud", to enable threshold-based soil moisture monitoring. Here the authors used cluster-based hierarchical SC and considered sensors with dynamic and automated recharge characteristics. Zervopoulos *et al.* [20] proposed a synchronization scheme for time correlated measurement in PA to reduce data transmissions, thereby minimizing the system's energy footprint and extending network lifetime. Wang *et al.* [21] surveyed various schemes that proposed the integration of mobile edge computing with SC to address the challenges of storage, trust, and data collection.

**(b) Virtual Sensor Provisioning Schemes:** In the literature, several schemes have been proposed (viz. [10]- [16]) for virtual sensor provisioning (VSP), a few of which are summarized here. Sarkar *et al.* [10] proposed a VSP scheme for WSN in which the virtual sensors are considered to be of two types – dormant and active, and are placed at the sink nodes. A sensor virtualization scheme is proposed [11], which

For personal use only

TABLE I: Comparison of DLSense with Other Virtual Sensor Provisioning Schemes

| Research Work | Architecture | Designed for | Energy Consumption | Message Overhead | Delay | Privacy |
|---|---|---|---|---|---|---|
| Sarkar, et al. [10] | Cloud-based | WSN | Yes | Yes | No | No |
| Gupta and Mukherjee [11] | Edge-based | WSN | Yes | No | Yes | No |
| Misra and Chakraborty [5], Zhang, et al. [12] | Cloud-based | SC | Yes | No | No | No |
| Dinh and Kim [13] | Cloud-based | SC | No | Yes | No | No |
| Al-Azez, et al. [14], Guijarro, et al. [15] | Cloud-based | IoT-services | Yes | No | Yes | No |
| Wang, et al. [16] | Edge-based | IoT-services | Yes | Yes | No | No |
| De Vita, et al. [17] | Edge-based | Iot-services | Yes | Yes | Yes | No |
| **Proposed Scheme DLSense** | Edge-based | SC | Yes | Yes | Yes | Yes |

considers distributed edge-based sensor mapping. On the other hand, optimal virtual sensor formation schemes are proposed in [5] that select an optimal subset of sensors to reduce resource consumption and ensure optimal load distribution in SC. In [12], the authors implemented a cluster-based VSP scheme to activate the most representative set of physical sensors for improving the service quality. An information centric network-based VSP scheme was proposed in [13], which used an external correlation-based prediction approach to predict sensor data at the cloud level. For IoT services, VSP schemes based on sensor mapping were proposed in [14], [15] while considering fault-tolerance, delay, and cost of service provisioning. Wang *et al.* [16] surveyed VSPs scheme for edge-based IoT-services platform using balance dynamics. Finally, Fabriz *et al.* [17] proposed *Deep Leaf*, a novel deep learning based quantization model for plant disease detection on the smart edge (IoT nodes) with improved accuracy memory utilization, energy consumption, and inference time.

***Synthesis:*** Table I qualitatively compares the proposed DLSense scheme with other existing works on VSP. Our synthesis of the literature leads to the following conclusions.

1) Most of the existing works on SC consider the formation of virtual sensors in the cloud, while using the edge layer for preliminary processing and storage.
2) Existing VSP schemes mostly focus on energy consumption and not on communication overhead and privacy.
3) Problems arising due to damaged or unemployed sensor nodes is not considered in general.

To address these issues, we design *DLSense*, a novel privacy-aware, edge-intelligence-empowered SC architecture that is equipped with smart sensor virtualization. The proposed scheme is presented in details in the following sections.

## III. Edge-empowered Sensor-Cloud Architecture

The proposed modified sensor-cloud architecture consists of the three primary layers of the traditional SC, namely the user layer, middle-ware layer, and physical sensor layer, as shown in Fig. 1. In the *user layer*, multiple users request for services to the SC service provider using web interfaces. The *middle-ware layer*, also known as the *virtual sensor layer*, configures virtual sensors to serve user requests. Unlike traditional SC, we consider the position of the *middle-ware layer* at the regional edge servers in the network edge, instead of the centralized cloud server. Due to processing in the edge,

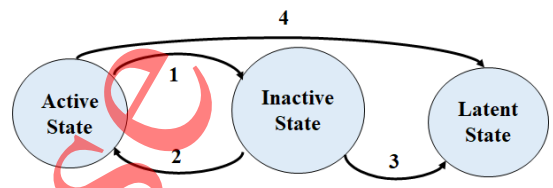


Fig. 2: Transition Diagram of Physical Sensor States

the huge amount of sensed data need not be sent to the cloud servers, thereby preventing communication bottlenecks and bandwidth hogging. Finally, the *physical sensor layer* consists of heterogeneous wireless sensor nodes deployed at various agricultural fields in different geographical regions. The edge nodes in the vicinity of sensor nodes in each field are responsible for collecting data and preliminary processing before sending them to the edge server.

Following [4], the main stakeholders of the proposed SC architecture are sensor owners, service providers, and end users. Let a set $\mathcal{O}$ of sensor owners, registered with a service provider $\mathcal{SP}$, serve a set $\mathcal{U}$ of end users. Additionally, the architecture comprises four entities, such as physical sensors, virtual sensors, edge nodes, and edge servers.

***Physical Sensor:*** A physical sensor $S_i \in \mathcal{S}$, where $\mathcal{S}$ denotes the set of sensor nodes, is defined using five-tuple $\langle S_i^{id}, S_i^{state}, S_i^{loc}, S_i^{\theta}, S^P \rangle$, where $S_i^{id}$ denotes the unique identifier; $S_i^{state}$ denotes the status, i.e., active, inactive, and latent (not deployed or damaged); $S_i^{loc}$ denotes the location; $S_i^{\theta}$ denotes the owner; and $S_i^P$ denotes the service provider associated with the sensor node $S_i$. Let $\mathcal{S}^{ac}$, $\mathcal{S}^{in}$, and $\mathcal{S}^{ln}$ denote the set of physical sensors in active, inactive, and latent states, respectively. The state transition diagram of a physical sensor node is shown in Fig. 2, in which the arrows numbered (1)-(4) depict the transitions. A sensor node in the active state transits into the inactive state when it is not in use (1), or into the latent state when it fails due to energy exhaustion (4). On the other hand, a sensor node in the inactive state transits into the active state when it is in use (2), or into the latent state when it is damaged or taken out of service (3). We assume that a latent sensor node cannot transit into any other state.

**Property 1.** *At a particular time instant, a physical sensor node cannot stay in more than one state. Thus, the intersection of the set of active, inactive, and latent nodes is null, i.e.,* $\mathcal{S}^{ac} \cap \mathcal{S}^{in} \cap \mathcal{S}^{ln} = NULL$.

***Virtual Sensor:*** A virtual sensor $V_x \in \mathcal{V}$, associ-

For Personal Use Only

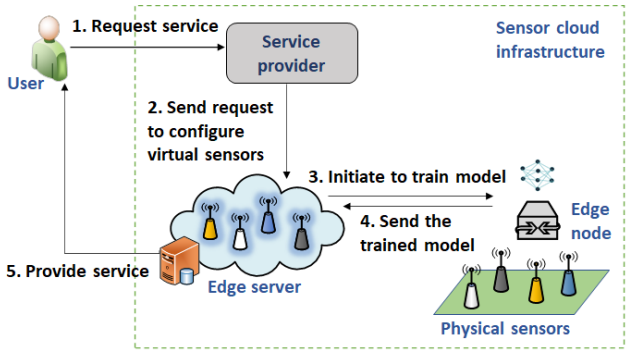


Fig. 3: Different steps of DLSense

ated with a physical sensor $S_i \in \mathcal{S}$, is represented as $\langle V_x^{id}, V_x^{type}, V_x^S, V_x^P, V_x^\phi \rangle$, where $V_x^{id}$ denotes the unique identifier; $V_x^{type}$ denotes the type of virtual sensor, i.e., active ($V_x^{ac}$), inactive ($V_x^{in}$), or latent ($V_x^{ln}$) based on the type of physical sensor it virtualizes; $V_x^S$ denotes the associated physical sensor node(s); $V_x^P$ denotes the related service provider; and $V_x^\phi$ denotes the associated service.

**Property 2.** *The mapping from the set of physical sensor nodes $\mathcal{S}$ to the set of virtual sensors $\mathcal{V}$ is onto.*

***Edge Node:*** An edge node $E_j \in \mathtt{E}$, where $\mathtt{E}$ denotes the set of edge nodes, associated with an agricultural field is represented as $\langle E_j^{id}, E_j^{loc}, E_j^\theta, E_j^S, E_j^V \rangle$, where $E_j^{id}$ denotes the unique identifier of the edge node, $E_j^{loc}$ its location, $E_j^\theta$ the associated sensor owner, $E_j^S$ the set of physical sensors, and $E_j^V$ the set of virtual sensors managed by the node.

**Property 3.** *The mapping from the set of physical sensor nodes $\mathcal{S}$ to the set of edge nodes $\mathtt{E}$ is not one to one.*

***Edge Server:*** An edge server $ES_l \in \mathtt{ES}$ where $\mathtt{ES}$ denotes the set of edge servers, configures the virtual sensors corresponding to each physical sensor in a specific region to fulfill the user demands. $ES_l$ is defined as $\langle ES_k^{id}, ES_k^{loc}, ES_k^\theta, ES_k^S, ES_k^V, ES_k^\phi \rangle$, where $ES_k^{id}$ denotes the unique identifier of the server, $ES_k^{loc}$ its location, $ES_k^P$ the set of service providers associated with the edge server, $ES_k^S$ the set of physical sensors deployed at a region, $ES_k^V$ the set of virtual sensors, and $ES_k^\phi$ the services provided by the edge server.

## IV. DLSense: Proposed Virtualization Scheme

This section introduces our novel virtual sensor formation scheme, *DLSense*, for the proposed modified SC architecture, in which virtualization takes place in the middle-ware layer comprising multiple regional edge servers. Fig. 3 depicts the different steps of service provisioning using DLSense. When a user $U_d$ requests the service provider $\mathcal{SP}$ for a service $\phi_z$ from region $R_p$, $\mathcal{SP}$ employs the regional edge server $ES_k$ to fulfill the service. In turn, $ES_k$ employs an edge node $E_j$ within $R_p$ for virtualization. This is known as the *selection phase*. Thereafter, in the *training phase*, $E_j$ initiates the training of a computation model using DLSense for each physical sensor $S_i$ used to fulfill the request. On completion of the training phase, $E_j$ sends the information of the trained model to the middle-ware layer to create the virtual sensor $V_x^i$ associated with $S_i$. Finally, in the *virtualization phase*, $ES_k$ delivers the service to the user with the help of $V_x^i$. The key notations used in this work are listed in Table II.

TABLE II: Key Notations

| Notations | Description |
|---|---|
| $\mathcal{S}^{ac}$, $\mathcal{S}^{in}$, $\mathcal{S}^{ln}$ | Set of active, inactive, and latent sensor nodes |
| $V_x^i$ | $x^{th}$ virtual sensor for physical sensor $S_i$ |
| $\rho_i$ | Datasize generated by $S_i$ |
| $C_i^r$, $C_i^m$, $C_i^d$ | Rent, maintenance, and deployment costs for $S_i$ |
| $\mathtt{E}$, $\mathtt{ES}$ | Set of edge nodes and servers, respectively |
| $\delta_{i,j}^T$, $\delta_{i,j}^P$ | Transmission & propagation delay from $S_i$ to $E_j$ |
| $\mathcal{E}_{i,j}^T$ | Transmission energy between $S_i$ and $E_j \in \mathtt{E}$ |
| $D_j$, $w_j$, $M_j$ | Data set, weight, and model size across $E_j$ |
| $w'$ | Aggregated weight |
| $\delta_j^c$, $\mathcal{E}_j^c$ | Computation delay and energy consumption at $E_j$ |
| $H_z$ | Message overhead for service $\phi_z$ |
| $C_j^{en}$,$C_j^{en,m}$,$u_j^{en}$ | Deployment, maintenance, and utilization of $E_j$ |
| $C_k^{es}$,$C_k^{es,m}$,$u_k^{es}$ | Deployment, maintenance, and utilization of $ES_k$ |

DLSense utilizes distributed learning for virtual sensor formation and introduces two types of virtualization schemes: (a) for active and inactive nodes, and (b) for latent nodes, as discussed in the following.

*1) Active and Inactive Node Virtualization:* Similar to [13], we design the virtualization scheme for active and inactive nodes based on correlation theory. Here, active node virtualization is performed using internal information correlation (IIC) based predictor, while inactive node virtualization is performed using hybrid (combination of internal and external information correlation) prediction model. However, contrary to [13], our proposed scheme performs the distributed model training phase in the edge nodes instead of the cloud, and the trained model is transmitted to the middle-ware layer to run the virtual sensors.

*2) Latent Node Virtualization:* DLSense uses cross correlation and distributed learning paradigm for latent node virtualization. When a service $\phi_z$ needs to be served by a sensor node $S_{i,p}^{ln}$ in the latent state in region $R_p$, it performs latent node virtualization process in the following three phases.

*Selection Phase:* Upon receiving the request, $ES_k$ searches for a region $R_q \in \mathcal{R}, \forall p \neq q$, having same type of sensor node as $S_{i,p}^{ln}$ in the active state i.e. $S_{i,q}^{ac}$. For each region $R_q \in \mathcal{R}$, the sensor nodes are represented as $\mathcal{S}_q$.

*Training Phase*: For all $S_{i,q}^{ac} \in \mathcal{S}_q$, where $type(S_{i,q}^{ac}) = type(S_{i,p}^{ln})$, $ES_k$ employs an edge node $E_j$ in $R_q$ to evaluate the training operation. Using cross correlation, $E_j$ first finds out the active companion (ACP) node $\{S_{i,q}^{ac}\}'$, which is highly correlated with the node $S_{i,q}^{ac}$. Thereafter, $E_j$ trains a model using a predefined ML algorithm to predict the value of $S_{i,q}^{ac}$ from the data of $\{S_{i,q}^{ac}\}'$. On completion of the training operation, each $E_j$ from $R_q \in \mathcal{R}, \forall p \neq q$, sends the trained model parameters to the $ES_k$, which aggregates the trained model parameters and learns the final model. However, if the accuracy of the global model is less than a threshold $0 \leq \sigma \leq 1$, the selection and training phases are repeated again.

For Personal Use Only

Let us consider that, in a single iteration, $N$ edge nodes participate during the training operation and store local sensor datasets. Each training data sample $a \in \mathcal{D}_j$ for edge node $E_j$ consists of two parts $(X_a, Y_a)$, where $X_a$ denotes the input vector and $Y_a$ denotes the corresponding output label. $E_j$ learns the model parameter $w_j$ on $\mathcal{D}_j$ using its corresponding loss function. For example, in case of linear regression, the loss function is the mean squared error calculated as $\sum_{a \in \mathcal{D}_j} (Y_a - \hat{Y}_a)^2/n$, where $n = |\mathcal{D}_j|$ and $\hat{Y}_a$ denotes the predicted data. After training, each edge node sends $w_j$ to $ES_k$ which performs the aggregation operation and evaluates the final model parameter as $w' = \sum_{j=1}^{N} w_j/N$.

*Virtualization Phase:* On completion of training operation, $ES_k$ designs a virtual sensor $V_x^i$ corresponding to $S_{i,p}^{ln}$ using the final model parameter $w'$, enabling the prediction of the value of $S_{i,p}^{ln}$ based on the data of other nodes within $R_p$.

Algorithm 1 describes different steps of the latent node virtualization. Its complexity mainly depends on the training operation of the ML model employed. We consider that each edge node performs the training operation in parallel. Thus, considering that linear regression model is used, the training time complexity is $O(n^2 d)$, where $n = |D_j|$ is the number of samples and $d$ denotes the dimension of the data.

**Algorithm 1** `Latent Node Virtualization`

**INPUTS:** $R_p, ES_k, E_j, S_{i,p}^{ln}, \sigma$
**OUTPUT:** $\{V_x^i\}$ corresponding to $\phi_z$
**PROCEDURE:**

1: $\mathcal{SP}$ requests $ES_k$ for $\phi_z$ using $S_{i,p}^{ln}$ in $R_p$
2: $ES_k$ searches $R_q \in \mathcal{R}, \forall p \neq q$ such that there exists $S_{i,q}^{ac}$ and type($S_{i,p}^{ln}$) == type($S_{i,q}^{ac}$)
3: **for** each $R_q \in \mathcal{R}$ **do**
4: $\quad ES_k$ finds $\{S_{i,q}^{ac}\}'$ highly correlated with $S_{i,q}^{ac}$
5: $\quad E_j$ learns $w_j$ using an ML algorithm
6: $\quad E_j$ sends $w_j$ to $ES_k$
7: **end for**
8: $ES_k$ evaluates $w' = \sum_{j=1}^{N} w_j/N$
9: **if** global accuracy reaches $\sigma$ **then**
10: $\quad ES_k$ creates virtual sensor $V_x^i$ using $w'$ to serve $\phi_z$
11: **else**
12: $\quad$ Go to Step 2 and repeat
13: **end if**
14: Design $\{V_x^i\}$ using $w'$

## V. Performance Evaluation

For performance evaluation, we compare the proposed modified SC architecture with the traditional one with respect to different performance metrics such as delay, resource consumption, service availability, and cost of service provisioning. We model these parameters as follows.

*1) Delay:* The delay incurred due to service provisioning in the proposed architecture primarily depends on virtualization. The total delay to virtualize node $S_i$ depends on three factors:

(a) *Communication delay* $\delta_{i,j}$ between the sensor node $S_i$ and edge node $E_j$ is calculated as $\delta_{i,j} = \max_{S_i \in \mathcal{S}^s}(\delta_{i,j}^T + \delta_{i,j}^P)$, where $\delta_{i,j}^T$ and $\delta_{i,j}^P$ respectively denote the transmission and propagation delays. Considering one hop communication, the transmission and propagation delays are formulated as: $\delta_{i,j}^T = \frac{\rho_i}{\tau_i}$ and $\delta_{i,j}^P = \frac{d_{i,j}\rho_i}{C}$, respectively, where $\rho_i$ denotes the data size generated by $S_i$; $\tau_i$ denotes the datarate of $S_i$; while $d_{i,j}$ denotes the distance between $S_i$ and $E_j$; and $C$ denotes the speed of light speed (constant). Let $\mathcal{S}^s$ be the set of nodes used for service provisioning.

(b) *Computation delay* $\delta_j^c$ in the edge node $E_j$ varies with the type of virtualization. For the active, inactive node, and latent nodes, the computation times are denoted as $\delta_j^{c,ac}$, $\delta_j^{c,in}$ and $\delta_j^{c,ln}$. We have,

$$\delta_j^c = (x \bmod 2)\delta_j^{c,ac} + (\lfloor x/2 \rfloor \bmod 2)\delta_j^{c,in} + (\lfloor x/4 \rfloor \bmod 4)\delta_j^{c,ln} \quad (1)$$

where $x$ is a ternary variable denoting the virtualization type: 1 for active, 2 for inactive, and 4 for latent.

(c) *Communication delay* $\delta_{j,k}$ between edge nodes and edge server $ES_k$ is calculated as $\delta_{j,k} = \max_{E_j \in \mathrm{E}^s}(\delta_{j,k}^T + \delta_{j,k}^P)$, where $\delta_{j,k}^T$ and $\delta_{j,k}^P$ denotes the transmission and propagation delays, respectively. Here, $\mathrm{E}^s$ denotes the set of edge nodes involved in provisioning the particular service.

Thus, the total delay incurred for virtualization in order to serve $\phi_z$ is given by $\delta_z^{DLSense} = \delta_{i,j} + \delta_{j,k} + \sum_{E_j \in E^s} \delta_j^c$.

Recall that, in traditional SC, the communication delay depends on the communication between the sensor and edge nodes, and between the edge nodes and cloud. Thus, the total communication delay to serve $\phi_z$ is $\delta_z^{sc} = \delta_{i,j} + \delta_{j,cs} + \delta_k^c$, where $\delta_{i,j}$ denotes the communication delay between $S_i$ and $E_j$; $\delta_{j,cs}$ denotes the communication delay between $E_j$ and the cloud server; and $\delta_k^c$ denotes the computation delay in the cloud. In this case, there is no computation delay at the edge nodes due to the transmission of raw sensed data.

*2) Energy Consumption:* The energy consumption of the proposed system depends on the following three factors.

(a) *Energy consumption $\mathcal{E}_{i,j}^T$ due to transmission of raw data from sensor node $S_i$ to edge node $E_j$*: Adopting the energy model proposed in [22], we calculate $\mathcal{E}_{i,j}^T = \mathcal{E}_i^{elec}\rho_i + \mathcal{E}_i^{amp}\rho_i d_{i,j}^2$, where $\mathcal{E}_i^{elec}$ and $\mathcal{E}_i^{amp}$ respectively denote the transmission and amplification energy of $S_i$; $\rho_i$ and $d_{i,j}$ are the size of data and distance between $S_i$ and $E_j$, respectively.

(b) *Energy consumption $\mathcal{E}_j^c$ due to computation at edge node $E_j$*: Following the model in [23], we calculate the computation energy at $E_j$ as $\mathcal{E}_j^c = \frac{\beta_j}{2}\eta_i\rho_i\sigma_j^2$, where $\beta_j$ and $\sigma_j$ denote the effective capacitance coefficient and CPU cycle frequency of $E_j$; whereas $\eta_i$ denotes CPU cycles needed to execute $\mathcal{D}_i$ and $\rho_i$ denotes the data sample size from $S_i$.

(c) *Energy consumption $\mathcal{E}_{j,k}^T$ due to communication of processed data between $E_j$ and $ES_k$*: Similar to (a), we calculate $\mathcal{E}_{j,k}^T = \mathcal{E}_j^{elec}\mathcal{M}_j + \mathcal{E}_j^{amp}\mathcal{M}_j d_{j,k}^2$, where $\mathcal{E}_j^{elec}$ and $\mathcal{E}_j^{amp}$ denote the transmission and amplification energy of $E_j$, respectively. The distance between $E_j$ and $ES_k$ is denoted by $d_{j,k}$ and $\mathcal{M}_j = \sum_{S_i \in \mathcal{S}^s} \mathcal{M}_{i,j}$, where $\mathcal{M}_{i,j}$ denotes the model size at $E_j$ for $S_i$.

Thus, using DLSense the total energy consumption for

For Personal Use Only

provisioning $\phi_z$ is:

$$\mathcal{E}_z^{DLSense} = \sum_{S_i \in \mathcal{S}^s} \sum_{E_j \in \mathbb{E}^s} (\mathcal{E}_{i,j}^T + \mathcal{E}_j^c + \mathcal{E}_{j,k}^T) \quad (2)$$

In contrast, in case of traditional SC, the total energy consumption for provisioning service $\phi_z$ is evaluated as:

$$\mathcal{E}_z^{sc} = \sum_{S_i \in \mathcal{S}^s} \sum_{E_j \in \mathbb{E}^s} (\mathcal{E}_{i,j}^T + \mathcal{E}_{j,k}^T), \quad (3)$$

where $\mathcal{E}_{i,j}^T$ and $\mathcal{E}_{j,k}^T$ denote the energy consumption due to the communication between $S_i$ and $E_j$, and between $E_j$ and the cloud, respectively. Here, the energy consumption for processing at the cloud is considered to be negligible.

*3) Message Overheads:* In the proposed architecture, the message overheads $H_z^{DLSense}$ for provisioning $\phi_z$ result from the transmission of sensed data from sensor nodes to the edge nodes, and trained model from edge nodes to the edge servers. Thus, $H_z^{DLSense} = \sum_{S_i \in \mathcal{S}^s} \rho_i + \sum_{E_j \in \mathbb{E}^s} \mathcal{M}_j$. In the traditional SC, the message overheads $H^{SC}$ for provisioning $\phi_z$ include data transmission from sensor nodes to the edge nodes, and from edge nodes to the cloud. It is calculated as: $H_z^{SC} = \sum_{S_i \in \mathcal{S}^s} \rho_i + \sum_{E_j \in \mathbb{E}^s} \sum_{S_i \in \mathcal{S}^s} \rho_i$.

*4) Service Availability:* Service availability is the ratio of the number of services served by the service provider to the total number of requests made by the users. In the proposed architecture, the service provider fails to provide a service if at least one of the following conditions hold. (i) The requested type of sensor node is not present in any of the regions in which the service provider operates; (ii) Maximum service handling capacity of the concerned edge nodes is exceeded; (iii) The delay threshold of the requested service cannot be met; and (iv) The service requires support from a latent node, and there is no correlation of the requested type of sensor data with other types of data sensed by its neighbouring nodes. On the other hand, in case of traditional SC, the service provider fails to provide a service if condition (ii) or (iii) holds, or if sensor nodes capable of serving the request are not available.

*5) Service Cost:* We model the service provisioning cost $C$ in terms of the following parameters.

(a) *Usage cost $C_i$ of sensor node $S_i$*: It consists of the rent $C_i^r$ paid to the sensor-owners, the maintenance cost $C_i^m$ per unit usage, and the redeployment cost $C_i^d$. We have $C_i = C_i^r + u_i C_i^m + y C_i^d$, where $u_i$ denotes the usage of node $S_i$ and $y$ denotes the number of redeployments of $S_i$. Thus, the total cost incurred $C^s$ is given as $C^s = \sum_{\forall S_i \in \mathcal{S}} C_i$.

(b) *Cost $C^e$ of edge infrastructure*: It includes the cost of deploying and maintaining edge nodes and servers. We have,

$$C^e = \sum_{\forall e_j} (C_j^{en} + u_j^{en} C_j^{en,m}) + \sum_{\forall ES_k} (C_k^{es} + u_k^{es} C_k^{es,m}) \quad (4)$$

where $u_j^{en}$, $C_j^{en}$, and $C_j^{en,m}$ denote the utilization, deployment [24], and maintenance costs of $E_j$, respectively. And $u_j^{es}$, $C_k^{es}$, and $C_k^{es,m}$ denote the utilization, deployment, and maintenance costs of $ES_k$, respectively. Thus, $C_z^{DLSense} = C^s + C^e$.

In traditional SC, the cost of provisioning $\phi_z$ is $C_z^{sc} = C^s + C^c$, where $C^c$ denotes the cost of using the cloud infrastructure [25]. We assume that $C^c >> C^e$ for all cases.

TABLE III: Simulation Parameters

| Parameters | Values |
|---|---|
| Simulation iterations | 100 |
| Number of service requests | 1-10 |
| Number of sensor nodes | 500, 1000, 1500 |
| Number of edge devices | 100 |
| Number of edge servers | 10 |
| Simulation area | $1 \times 1km^2$ |
| Amount of sensed data | 47, 146 Kb |
| Communication range of sensor nodes | 100m |
| Communication range of edge nodes | 350m |
| CPU deception power of edge nodes | 6.5 W/sec |
| Transmitter Electronics ($E_{elec}$ ) | 50 nJ/bit [22] |
| Transmit Amplifier ($E_{amp}$ ) | 100 pJ/bit/$m^2$ [22] |

## A. Simulation Settings and Benchmarks

We designed a simulation environment using MATLAB 2018a. We considered random deployment of heterogeneous sensor nodes in an agricultural land of $1 \times 1$ km$^2$ area consisting of square fields of $100 \times 100$ m$^2$ area, each belonging to different farmers. These are used to serve agricultural services such as irrigation control, crop Nitrogen status estimation, and water stress assessment. The simulation parameters are listed in Table III. We compared the performance of the proposed edge-based SC architecture with the traditional cloud-based SC [4]. For comparison, we consider two variants of the proposed architecture: (i) edge-based SC implementing simple aggregation in the edge layer, and (ii) incorporating DLSense in the edge layer. In edge-based SC, the calculation of delay and energy consumption for communication and net profit earned is the same as those in DLSense, while the calculation of service availability is the same as that in cloud-based SC. We simulated the three schemes with varying number of sensor nodes for a fixed number of edge nodes and services.

## B. Results and Discussions

From Fig. 4(a), we observe that DLSense reduces latency by 81% compared to the cloud-based SC because of the position of virtual sensors closer to the users. This also results from the fact that DLSense removes the need to send raw data from the edge devices across the network, by enabling the sharing of the trained model. We also observe similar performance for DLSense and edge-based SC due to the reduction in delay through the sharing of aggregated data and the reduction in computations in the edge devices.

Fig. 4(b) depicts that the energy consumption is significantly less in case of DLSense and edge-based SC compared to the cloud-based SC. This is because the energy consumption primarily depends on computations and data transmission in the edge and sensor nodes. The transmission energy consumption is significantly higher in cloud-based SC due to the transmission of huge volumes of sensed data across large distances to the cloud. On the other hand, in DLSense, although the computation energy consumption in edge devices is higher, it is insignificant as compared to the transmission energy. Furthermore, in edge-based SC, the computational energy consumption is less due to the use of aggregation. However,

For Personal Use Only

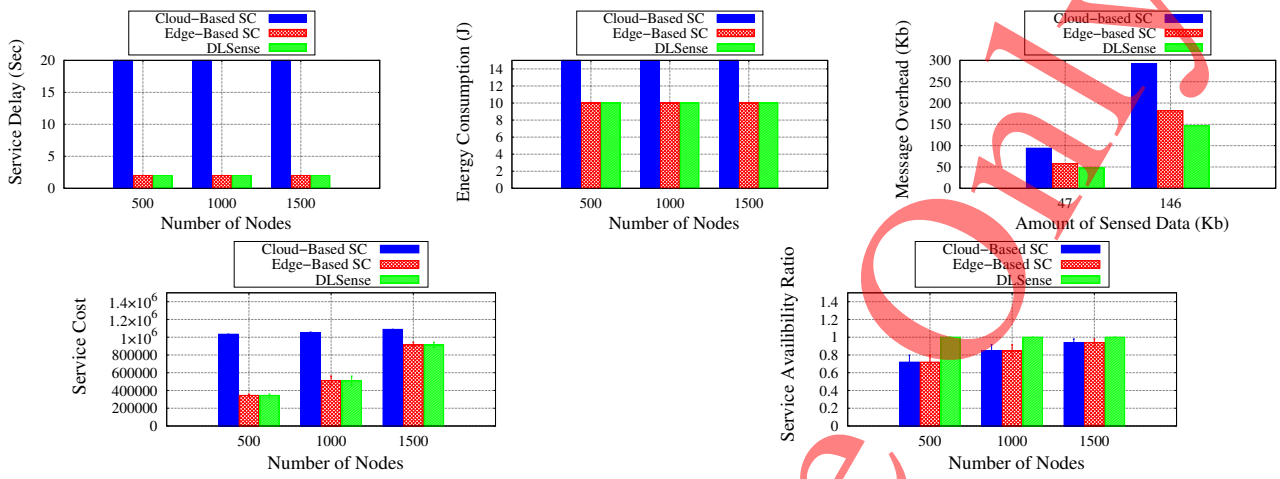


Fig. 4: Comparison of Results (a) Service Delay, (b) Energy Consumption, (c) Message Overheads, (d) Service Cost, and (e) Service Availability – vs. Number of Nodes

this decrease is compensated by an increase in transmission energy to transmit aggregated data to the edge servers.

Fig. 4(c) demonstrates that DLSense outperforms the other two schemes in terms of message overheads. This is due to the fact that the transmission of sensed data is limited to between the sensor nodes and their nearest edge nodes. Beyond the edge nodes, only the trained model information is shared with the edge server to instantiate and run the virtual sensors to serve the end-users. On the other hand, cloud-based SC requires the transmission of huge amount of raw sensed data to the cloud, while edge-based SC requires the transmission of aggregated sensor data to edge servers to virtualize a sensor node.

From Fig. 4(d), we observe that the service cost using DLSense is 16-66% higher than cloud-based and is similar to the edge-based SC. This is due to the huge maintenance cost of cloud servers compared to the edge infrastructure. Additionally, from Fig. 4e we observe that DLSense exhibits 7-39% higher service availability than the existing schemes because, even in the absence of active nodes, DLSense is able to serve the user requests using smart virtualization unlike the other two schemes. From these two figures, we infer that the profit earned by service provider is highest using DLSense.

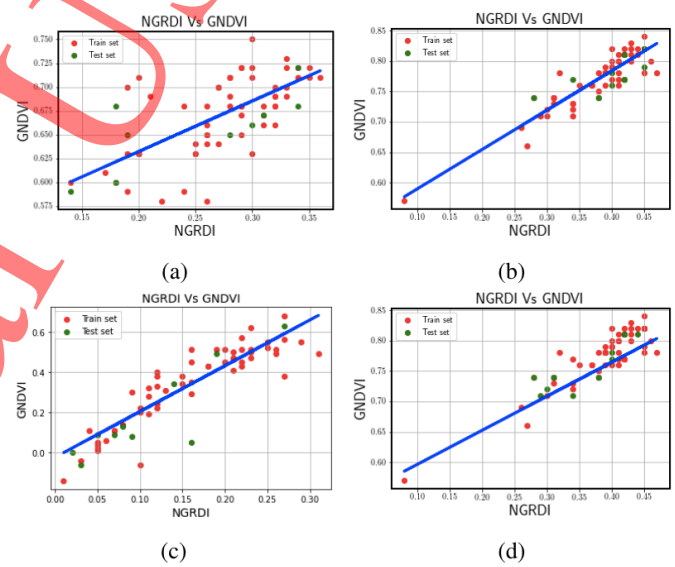


Fig. 5: Training model of (a) field 1 with $R^2 = 0.55$, (b) field 2 with $R^2 = 0.91$, and (c) field 3 with $R^2 = 0.91$. (d) Aggregation model with $R^2 = 0.80$

### C. Case Study of a Typical PA Application

For a simple case study, we considered a PA application of remote sensing-based crop (paddy) nitrogen status estimation on four different agricultural fields. Towards this aim, application-specific sensors, such as unmanned aerial vehicles (UAVs) equipped with multi-spectral camera were deployed on Fields 1-3 and UAV equipped with RGB camera was deployed on Field 4. These sensors acquire aerial images from which different vegetation indices were extracted and the index most correlated with the ground truth data (chlorophyll content using SPAD meter, in this case) was selected. We consider a scenario in which the user of Field 4 is interested in the service of a multispectral camera. In such a case, using traditional SC, the service provider is required to conduct a flight using a UAV equipped with multispectral camera to Field 4, which may not be feasible and cost-effective. However, using DLSense, the service provider is able to generate the multispectral sensor data (which acts as the latent node) for Field 4. To demonstrate this, we experimented by deploying the system as mentioned above and acquired the real sensor data. Using DLSense, a linear regression model is trained for Fields 1-3 to predict the value of a multispectral index (such as GNDVI) from the correlated RGB index (such as NGRDI). Figs. 5(a)-(c) show the trained models using regression for Fields 1-3, respectively and Fig. 5d shows the aggregation model for Field 4. The statistical measure R-squared ($R^2$) value for the trained models at Field 1, Field 2, and Field 3 score 0.55, 0.91, and 0.91, respectively; and the aggregated model for Field 4 scores 0.80 which represents high level of correlation between the dependent variable (GNDVI) and independent variable (NGRDI).

For Personal Use Only

## VI. Conclusion

In this paper, we described an edge-based sensor-cloud (SC) architecture equipped with smart sensing for precision agriculture (PA) applications. We proposed a distributed learning-based virtualization scheme, called DLSense, which utilizes correlation theory and ML models for generating virtual sensors for active, inactive, and latent nodes. We evaluated the performance of the proposed architecture and compared it with traditional SC through simulation experiments. We observed that the proposed scheme decreases energy consumption, latency, and message overheads, and service cost. Additionally, we presented a case study of an agricultural application to demonstrate the applicability of the proposed scheme.

This work can be extended in future to study the economics of the proposed architecture in comparison with the existing economic models for SC. We also plan to study the dynamics introduced due to the presence of edge infrastructure providers. Another possible future direction is to evaluate and compare the performance of different ML models in the context of various PA applications.

## Acknowledgments

The authors are grateful to the guest editors and anonymous reviewers for insightful comments that helped improve the quality of the manuscript significantly. This research is supported by the scheme of IMPacting Research, INnovation and Technology (IMPRINT) (Project no. 5682) sponsored by the Ministry of Human Resources Development, Government of India, New Delhi and by the Ministry of Agriculture and Farmers' Welfare, Government of India, New Delhi (Coordinated by Director, Mahalanobis National Crop Forecast Centre, New Delhi). The work of S. K. Das is supported by the VAJRA Faculty Scheme under the Science and Engineering Research Board (SERB), India and the US National Science Foundation grant SCC-1952045 on "Smart Integrated Farm Network for Rural Agricultural Communities (SIRAC)."

## References

[1] T. Ojha, S. Misra, and N. S. Raghuwanshi, "Wireless sensor networks for agriculture: The state-of-the-art in practice and future challenges," *Comput. and Electron. in Agriculture*, vol. 118, pp. 66–84, 2015.

[2] M. A. Zamora-Izquierdo, J. Santa, J. A. Martínez, V. Martínez, and A. F. Skarmeta, "Smart farming IoT platform based on edge and cloud computing," *Biosystems engineering*, vol. 177, pp. 4–17, 2019.

[3] A. Lytos, T. Lagkas, P. Sarigiannidis, M. Zervakis, and G. Livanos, "Towards smart farming: Systems, frameworks and exploitation of multiple sources," *Computer Networks*, vol. 172, p. 107147, 2020.

[4] S. Misra, S. Chatterjee, and M. S. Obaidat, "On theoretical modeling of sensor cloud: A paradigm shift from wireless sensor network," *IEEE Systems J.*, vol. 11, no. 2, pp. 1084–1093, 2014.

[5] S. Misra and A. Chakraborty, "Qos-aware dispersed dynamic mapping of virtual sensors in sensor-cloud," *IEEE Trans. Serv. Comput.*, 2019.

[6] X. Ma, S. Wang, S. Zhang, P. Yang, C. Lin, and X. S. Shen, "Cost-efficient resource provisioning for dynamic requests in cloud assisted mobile edge computing," *IEEE Trans. Cloud Comp.*, 2019.

[7] L. Pu, L. Jiao, X. Chen, L. Wang, Q. Xie, and J. Xu, "Online resource allocation, content placement and request routing for cost-efficient edge caching in cloud radio access networks," *IEEE J. Sel. Areas in Commun.*, vol. 36, no. 8, pp. 1751–1767, 2018.

[8] T. N. Gia, L. Qingqing, J. P. Queralta, Z. Zou, H. Tenhunen, and T. Westerlund, "Edge AI in smart farming IoT: CNNs at the edge and fog computing with LoRa," in *Proc. IEEE AFRICON*, 2019.

[9] R. S. Alonso, I. Sittón-Candanedo, Ó. García, J. Prieto, and S. Rodríguez-González, "An intelligent Edge-IoT platform for monitoring livestock and crops in a dairy farming scenario," *Ad Hoc Networks*, vol. 98, p. 102047, 2020.

[10] C. Sarkar, V. S. Rao, R. V. Prasad, S. N. Das, S. Misra, and A. Vasilakos, "Vsf: An energy-efficient sensing framework using virtual sensors," *IEEE Sensors J.*, vol. 16, no. 12, pp. 5046–5059, 2016.

[11] A. Gupta and N. Mukherjee, "A cloudlet platform with virtual sensors for smart edge computing," *IEEE IoT J.*, vol. 6, no. 5, pp. 8455–8462, 2019.

[12] M.-Z. Zhang, L.-M. Wang, and S.-M. Xiong, "Using machine learning methods to provision virtual sensors in sensor-cloud," *Sensors J.*, vol. 20, no. 7, p. 1836, 2020.

[13] T. Dinh and Y. Kim, "Information centric sensor-cloud integration: An efficient model to improve wireless sensor networks' lifetime," in *Proc. IEEE ICC*, 2017, pp. 1–6.

[14] Z. T. Al-Azez, A. Q. Lawey, T. E. H. El-Gorashi, and J. M. H. Elmirghani, "Energy efficient iot virtualization framework with peer to peer networking and processing," *IEEE Acc.*, vol. 7, pp. 50 697–50 709, 2019.

[15] L. Guijarro, V. Pla, J. Vidal, and M. Naldi, "Game theoretical analysis of service provision for the internet of things based on sensor virtualization," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 3, pp. 691–706, 2017.

[16] T. Wang, G. Zhang, A. Liu, M. Z. A. Bhuiyan, and Q. Jin, "A secure iot service architecture with an efficient balance dynamics based on cloud and edge computing," *IEEE IoT J.*, vol. 6, no. 3, pp. 4831–4843, 2019.

[17] F. De Vita, G. Nocera, D. Bruneo, V. Tomaselli, D. Giacalone, , and S. K. Das, "Quantitative analysis of deep leaf: a plant disease detector on smart edge," *IEEE Conference on Smart Computing (SMARTCOMP)*, pp. 49–56, 2020.

[18] T. Ojha, S. Misra, and N. S. Raghuwanshi, "Sensing-cloud: Leveraging the benefits for agricultural applications," *Comp. and Elec. in Agriculture*, vol. 135, pp. 96 – 107, 2017.

[19] S. Tyagi, M. S. Obaidat, S. Tanwar, N. Kumar, and M. Lal, "Sensor cloud based measurement to management system for precise irrigation," in *Proc. IEEE GLOBECOM*, 2017, pp. 1–6.

[20] A. Zervopoulos, A. Tsipis, A. G. Alvanou, K. Bezas, A. Papamichail, S. Vergis, A. Stylidou, G. Tsoumanis, V. Komianos, G. Koufoudakis *et al.*, "Wireless sensor network synchronization for precision agriculture applications," *Agriculture*, vol. 10, no. 3, p. 89, 2020.

[21] T. Wang, Y. Lu, Z. Cao, L. Shu, X. Zheng, A. Liu, and M. Xie, "When sensor-cloud meets mobile edge computing," *Sensors J.*, vol. 19, no. 23, p. 5324, 2019.

[22] W. R. Heinzelman, A. Chandrakasan, and H. Balakrishnan, "Energy-efficient communication protocol for wireless microsensor networks," in *Proc. 33$^{rd}$ IEEE Annu. Hawaii Int. Conf. on system Sci.*, 2000.

[23] T. D. Burd and R. W. Brodersen, "Processor Design for Portable Systems," *J. of VLSI Signal Process. Syst. for Signal, Image and Video Technol.*, vol. 13, no. 2-3, pp. 203–221, 1996.

[24] A. Triantafyllou, P. Sarigiannidis, S. Bibi, F. Vakouftsi, and P. Vassilis, "Modelling deployment costs of Precision Agriculture Monitoring Systems," in *Proc. 16$^{th}$ IEEE DCOSS*, 2020, pp. 252–259.

[25] S. El Kafhali and K. Salah, "Modeling and analysis of performance and energy consumption in cloud data centers," *Arabian Journal for Science and Engineering*, vol. 43, no. 12, pp. 7789–7802, 2018.

For Personal Use Only